UNIVERSIDAD NACIONAL DE SAN AGUSTÍN

FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



CONSTRUCCIÓN DE SOFTWARE

"Modelo C4 de RutaColca"

Integrantes:

Estrada Arce Sergio Emilio
Mamani Quispe Renzo Geomar
Yana Huanca Christian Alexander
Estefanero Palma Rodrigo

Profesor:
Mg. Antonio Arroyo Paz

AREQUIPA – PERÚ
2026

1. Propósito del trabajo

En clase estudiamos los siete diagramas del modelo C4 usando el caso AgroConecta. En este trabajo aplicamos lo mismo a un sistema distinto, RutaColca, una plataforma para reservar tours al valle del Colca. El resultado es un modelo C4 escrito como código en Structurizr DSL, del cual salen todos los diagramas de este documento.

Dividimos el trabajo en dos partes. La Parte A se hizo en los 25 minutos de clase y cubre el inventario de elementos, el diagrama de contexto, el de contenedores y el dinámico de «Reservar y pagar un tour». La Parte B es la extensión: despliegue en producción, el reto de reservas sin conexión en Chivay, la exportación a otras notaciones y la bitácora de uso de IA.

Tabla 1. Estructura del trabajo y dónde está cada parte en este documento.

Parte	Qué se produce	Secciones
A. Modelado en clase	Inventario, contexto, contenedores, dinámico, revisión cruzada y explicación para una persona no técnica	4 a 9
B. Extensión	Despliegue, reto sin conexión (ADR-002), exportación a C4-PlantUML y Mermaid, bitácora de IA	10 a 13

2. Organización del grupo

Cada integrante asumió un rol. En la Parte B los roles rotaron para que los cuatro escribiéramos DSL al menos una vez, como pedía la guía.

Tabla 2. Roles del grupo y evidencia de cada uno.

Integrante	Rol (Parte A)	Responsabilidad	Evidencia
Rodrigo Estefanero	Modelador	Decidió qué elementos y relaciones entran en cada diagrama y dibujó el borrador en la pizarra.	Inventario C4 (sección 4)
Renzo Mamani	Redactor del DSL	Escribió workspace.dsl, lo validó y renderizó las vistas.	Archivo .dsl (Anexo B)
Christian Yana	Revisor	Aplicó la lista de revisión y condujo la revisión cruzada.	Tabla de hallazgos (sección 8)
Sergio Estrada	Vocero	Explicó el contexto a una persona no técnica y llevó la bitácora de prompts.	Explicación (sección 9) y bitácora (sección 13)

Tabla 3. Rotación de roles en la Parte B.

Paso	Quién escribió el DSL o el documento	Qué hizo
7. Despliegue	Rodrigo Estefanero	Nodos, instancias, balanceador y réplicas de la API.
8. Reto sin conexión	Christian Yana	Contenedor nuevo, relaciones nuevas y vista ContenedoresReto.
9. Exportación	Sergio Estrada	Estilos y etiquetas del DSL, comandos de la CLI y comparación de notaciones.
8. ADR-002	Renzo Mamani	Redacción del ADR-002 y validación final del modelo.

> **Reglas de trabajo que seguimos**

> **La IA propone, el equipo decide y verifica.** Todo lo que sugirió el asistente se revisó con la lista del Anexo A y quedó anotado en la bitácora.
>
> **El modelo es la fuente única.** Los diagramas salen de workspace.dsl; no se dibujan a mano después.
>
> **Privacidad.** No pegamos datos personales, credenciales ni datos reales de agencias o turistas en los prompts.

3. El caso: RutaColca

El valle del Colca está a unas cuatro horas de Arequipa y recibe miles de visitantes al año. Hoy las agencias de Arequipa y Chivay venden sus tours por teléfono y redes sociales, y coordinan a los guías con mensajes sueltos. Un consorcio de agencias quiere RutaColca, una plataforma donde las agencias publican sus tours con fechas de salida y cupos, los turistas reservan y pagan en línea, y cada reserva se confirma solo después de validar el boleto turístico del Colca con AUTOCOLCA. Los guías no usan la plataforma: reciben el itinerario y la lista del grupo por WhatsApp. Un administrador aprueba a las agencias nuevas y revisa las ventas.

La integración con AUTOCOLCA es un supuesto didáctico del caso, así que la tratamos como un sistema externo con una API HTTPS.

3.1 Historias de usuario del MVP

Tabla 4. Historias de usuario de RutaColca.

ID	Historia de usuario	Prioridad
HU-01	Como operador de agencia, quiero publicar un tour con sus fechas de salida y cupos para venderlo en línea.	Alta
HU-02	Como turista, quiero reservar una salida y pagarla en línea para asegurar mi lugar.	Alta
HU-03	Como turista, quiero registrar el número de mi boleto turístico para que se valide antes de confirmar.	Alta
HU-04	Como guía, quiero recibir por WhatsApp el itinerario y la lista del grupo un día antes de la salida.	Media
HU-05	Como operador de una agencia en Chivay, quiero registrar reservas presenciales aunque no haya internet.	Media (reto)
HU-06	Como administrador, quiero aprobar agencias y ver las ventas por agencia y por mes.	Baja

3.2 Restricciones y decisiones ya tomadas

Tabla 5. Restricciones del caso.

Tipo	Descripción
Plazo y equipo	MVP en 1 mes con 4 desarrolladores, apoyados por asistentes de IA.
ADR-001 (tecnología)	Web del turista en React (SPA); portal de agencias en React como PWA; API en Java 21 con Spring Boot; PostgreSQL 16; despliegue en un proveedor cloud con Docker.
Pagos	Se usa una pasarela externa; RutaColca no guarda datos de tarjetas.
Conectividad	En Chivay la conexión es intermitente (origen del reto de la Parte B).
Demanda	Picos en temporada alta (julio y agosto) y en feriados largos.

Parte A · Modelado en clase

4. Paso 1: inventario de elementos C4

Antes de dibujar clasificamos cada elemento del caso según las abstracciones de C4: persona, sistema de software (propio o externo) o contenedor. Para los contenedores nos guiamos por la definición de Brown: una aplicación o un almacén de datos que debe estar en ejecución para que el sistema funcione. Esto no tiene relación con los contenedores de Docker. Siglas usadas en este documento: SPA (Single Page Application, aplicación web de una sola página), PWA (Progressive Web App, aplicación web instalable que puede trabajar sin conexión) y AUTOCOLCA (la entidad que emite el boleto turístico del Colca, que en este caso modelamos como sistema externo).

Tabla 6. Inventario C4 de RutaColca.

Elemento del caso	Tipo C4	Justificación
Turista	Persona	Es un humano que busca, reserva y paga tours en la plataforma.
Operador de agencia	Persona	Es un rol humano que publica tours y registra reservas.
Guía de turismo	Persona	Es un humano que forma parte del flujo: recibe el itinerario, aunque sea por WhatsApp y no entre a la plataforma.
Administrador	Persona	Es un rol humano que aprueba agencias y revisa ventas.
RutaColca	Sistema de software (propio)	Es el sistema que construye el equipo y entrega valor al consorcio.
Pasarela de pagos	Sistema externo	Cobra con tarjeta o billetera móvil; lo opera un tercero.
WhatsApp Business	Sistema externo	Mensajería de un tercero que usamos para avisar a los guías.
Sistema de AUTOCOLCA	Sistema externo	Emite y valida el boleto turístico; lo opera otra entidad.
Web del turista	Contenedor	Aplicación (React SPA) que debe estar en ejecución para que el turista reserve.
Portal de agencias	Contenedor	Aplicación (React PWA) para operadores y administrador.
API	Contenedor	Aplicación (Java 21, Spring Boot) con las reglas de negocio.
Base de datos	Contenedor	Almacén de datos (PostgreSQL 16) con tours, salidas y reservas.

> **Decisión del grupo sobre el guía**
> Discutimos si el guía debía aparecer en el contexto, ya que no usa la plataforma. Decidimos incluirlo como persona porque el sistema existe en parte para darle información: sin el guía el flujo de la reserva queda incompleto. Como solo habla con WhatsApp, la flecha sale de WhatsApp Business hacia el guía, y no desde RutaColca, para mostrar que el contacto es indirecto.

5. Paso 2: diagrama de contexto

Ubicamos RutaColca como una sola caja, con las cuatro personas y los tres sistemas externos con los que se comunica. Cada relación se lee como una oración, por ejemplo «Turista busca tours, reserva y paga en RutaColca». En este nivel no mostramos protocolos, porque eso se ve en el diagrama de contenedores.

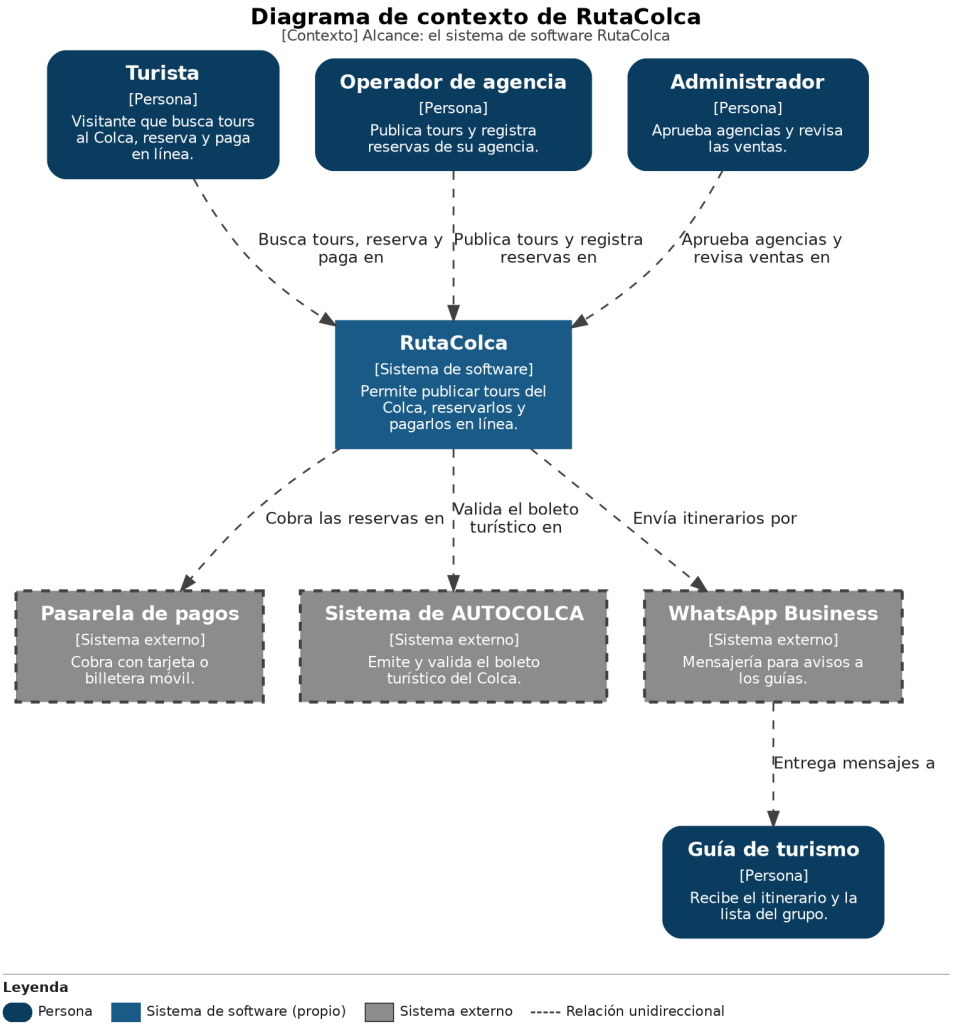


Figura 1. Diagrama de contexto de RutaColca (generado del modelo en Structurizr DSL).

Tabla 7. Relaciones del diagrama de contexto.

Origen	Relación	Destino
Turista	Busca tours, reserva y paga en	RutaColca
Operador de agencia	Publica tours y registra reservas en	RutaColca
Administrador	Aprueba agencias y revisa ventas en	RutaColca
RutaColca	Cobra las reservas en	Pasarela de pagos
RutaColca	Valida el boleto turístico en	Sistema de AUTOCOLCA
RutaColca	Envía itinerarios por	WhatsApp Business
WhatsApp Business	Entrega mensajes a	Guía de turismo

Verificación del contexto

Punto de la guía	Resultado	Cómo lo comprobamos
RutaColca es un solo rectángulo, sin base de datos ni frontend dentro	✔ Cumple	En la Figura 1 no hay ningún elemento dentro de la caja.
Están las cuatro personas	✔ Cumple	Turista, operador, guía y administrador.
Los tres sistemas externos aparecen en gris y con descripción	✔ Cumple	Pasarela, AUTOCOLCA y WhatsApp Business, con borde discontinuo para que se distingan en blanco y negro.

Punto de la guía	Resultado	Cómo lo comprobamos
Cada flecha tiene un verbo específico y una sola dirección	✔ Cumple	No usamos la palabra «usa» ni flechas dobles.
El diagrama tiene título y leyenda	✔ Cumple	Título arriba y leyenda al pie de la figura.

6. Paso 3: diagrama de contenedores

Hicimos zoom a RutaColca: cambiamos la caja por su límite y pusimos adentro las aplicaciones y los almacenes de datos que fija el ADR-001. Cada contenedor lleva su tecnología y cada relación entre contenedores lleva su protocolo. Las relaciones del contexto pasaron al contenedor que realmente se comunica; por ejemplo, quien cobra en la pasarela es la API, no el sistema completo.

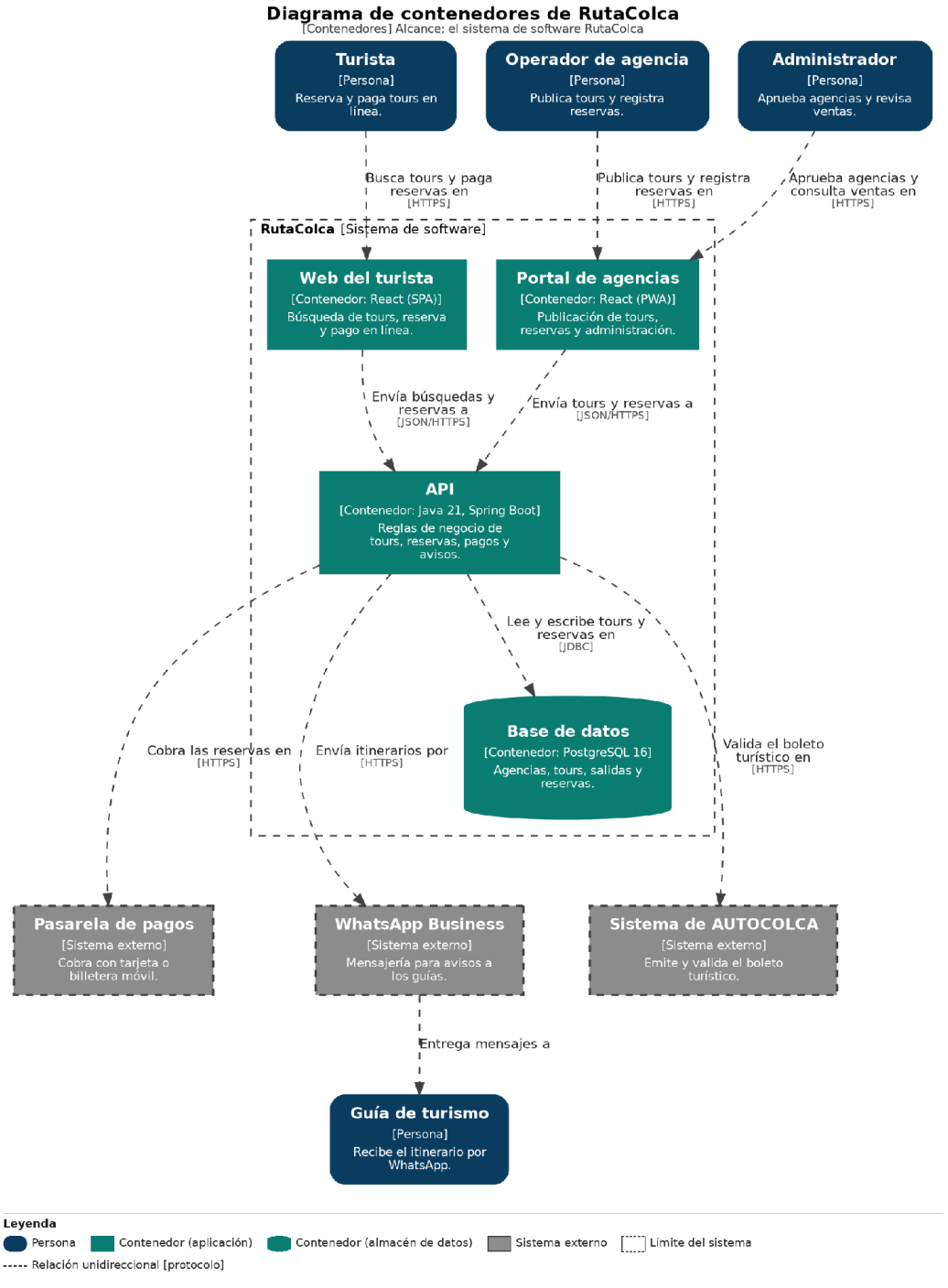


Figura 2. Diagrama de contenedores de RutaColca.

Tabla 8. Contenedores de RutaColca.

Contenedor	Tipo	Tecnología	Responsabilidad
Web del turista	Aplicación	React (SPA)	Búsqueda de tours, reserva y pago en línea.
Portal de agencias	Aplicación	React (PWA)	Publicación de tours, registro de reservas y administración (aprobar agencias, ver ventas).
API	Aplicación	Java 21, Spring Boot	Reglas de negocio de tours, reservas, pagos y avisos; habla con los sistemas externos.
Base de datos	Almacén de datos	PostgreSQL 16	Agencias, tours, salidas y reservas.

Tabla 9. Relaciones del diagrama de contenedores.

Origen	Relación	Destino	Protocolo
Turista	Busca tours y paga reservas en	Web del turista	HTTPS
Operador de agencia	Publica tours y registra reservas en	Portal de agencias	HTTPS
Administrador	Aprueba agencias y consulta ventas en	Portal de agencias	HTTPS
Web del turista	Envía búsquedas y reservas a	API	JSON/HTTPS
Portal de agencias	Envía tours y reservas a	API	JSON/HTTPS
API	Lee y escribe tours y reservas en	Base de datos	JDBC
API	Cobra las reservas en	Pasarela de pagos	HTTPS
API	Valida el boleto turístico en	Sistema de AUTOCOLCA	HTTPS
API	Envía itinerarios por	WhatsApp Business	HTTPS
WhatsApp Business	Entrega mensajes a	Guía de turismo	—

La relación de WhatsApp hacia el guía no lleva protocolo porque queda fuera de lo que construimos: es un servicio de terceros que habla con el teléfono de una persona.

Verificación de contenedores

Punto de la guía	Resultado	Cómo lo comprobamos
Hay al menos cuatro contenedores	✔ Cumple	Web del turista, portal de agencias, API y base de datos.
Cada contenedor indica tecnología	✔ Cumple	React (SPA), React (PWA), Java 21 Spring Boot y PostgreSQL 16.
Ningún contenedor se llama Docker, VM o servidor	✔ Cumple	Docker aparece solo como tecnología de un nodo en el despliegue.
Las relaciones salen del contenedor correcto y llevan protocolo	✔ Cumple	Ver Tabla 9.
Los sistemas externos y las personas del contexto siguen presentes	✔ Cumple	Las cuatro personas y los tres sistemas externos están en la Figura 2.

7. Paso 4: diagrama dinámico «Reservar y pagar un tour»

El diagrama dinámico cuenta una sola historia, la de HU-02 y HU-03, con los pasos numerados. Structurizr solo acepta pasos que correspondan a relaciones ya declaradas en el modelo estático; por eso cada flecha de la Figura 3 sale de una relación de la Tabla 9.

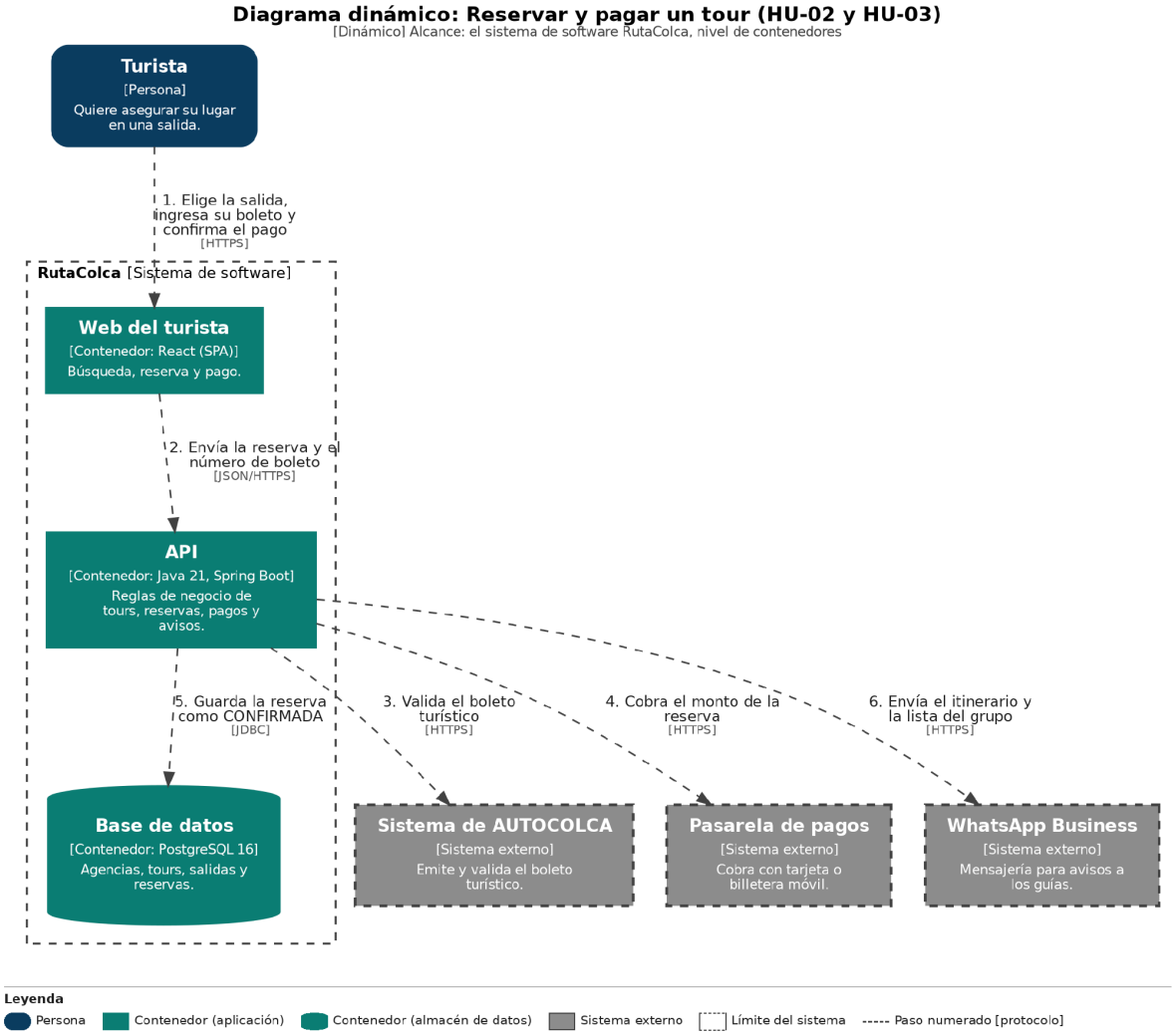


Figura 3. Diagrama dinámico: el turista reserva y paga un tour.

Tabla 10. Pasos del escenario.

Paso	Qué ocurre	Protocolo
1	El turista elige la salida, ingresa el número de su boleto y confirma el pago en la web.	HTTPS
2	La web envía la reserva y el número de boleto a la API.	JSON/HTTPS
3	La API valida el boleto turístico en AUTOCOLCA.	HTTPS
4	La API cobra el monto de la reserva en la pasarela de pagos.	HTTPS
5	La API guarda la reserva como CONFIRMADA en la base de datos.	JDBC
6	La API envía el itinerario y la lista del grupo por WhatsApp Business.	HTTPS

Decisión de diseño: ¿validar el boleto antes o después de cobrar?

Decidimos **validar el boleto antes de cobrar** (paso 3 antes del paso 4). Si el boleto no es válido, la reserva no se puede confirmar según HU-03, y cobrar primero obligaría a devolver el dinero. Validar antes evita cobros que luego hay que revertir.

La numeración de la Figura 3 refleja esta decisión. Para mantener un solo escenario, el paso 6 dibuja el envío del itinerario al final de la historia; la programación de ese aviso un día antes de la salida (HU-04) la resuelve la API y no cambia el diagrama.

Con respecto a los datos de tarjeta: RutaColca solo recibe de la pasarela el resultado del cobro, nunca el número de la tarjeta. Es un supuesto que asumimos de la restricción de pagos del caso.

Al escribir la vista no nos faltó ninguna relación en el modelo estático, así que no hubo que agregar nada.

8. Paso 5: revisión cruzada

Intercambiamos el modelo con otro grupo y el revisor, Christian Yana, aplicó la lista del Anexo A. Antes del intercambio también pasó nuestro workspace.dsl por un asistente de IA con el prompt P5 (auditor C4) y verificó cada hallazgo contra el modelo antes de aceptarlo. La Tabla 11 resume los hallazgos sobre nuestro modelo.

Tabla 11. Registro de hallazgos de la revisión cruzada.

N.º	Hallazgo	¿Lo aceptamos?	Motivo
1	Faltaba el guía como persona del contexto.	Sí	Recibe información del sistema por WhatsApp; es actor del contexto.
2	En el contexto se dibujó la base de datos dentro de RutaColca.	No	En contexto el sistema es una sola caja; la BD es un contenedor.
3	La relación portal → API decía «usa».	Sí, se corrigió	Se cambió a «Envía tours y reservas a» y se agregó el protocolo JSON/HTTPS.
4	Se propuso agregar Kafka y Redis.	No	No están en el ADR-001; no se inventan tecnologías.
5	La relación con la pasarela estaba en dos direcciones.	Sí, se corrigió	Se dejó una sola dirección: la API cobra en la pasarela.
6	Faltaba el protocolo en API → base de datos.	Sí, se corrigió	Se agregó el protocolo JDBC.

En el Anexo A aplicamos la lista completa a cada diagrama.

9. Paso 6: explicar el contexto a una persona no técnica

El vocero, Sergio Estrada, explica en dos minutos el diagrama de contexto como si hablara con la gerente de una agencia de Chivay. El borrador salió del prompt P6 y el grupo lo corrigió con sus propias palabras; la versión final es del vocero.

> **Explicación para la gerente de una agencia de Chivay (123 palabras)**
> RutaColca es una página web donde su agencia puede publicar sus tours al Colca con las fechas de salida y los cupos disponibles. Los turistas entran, eligen una salida y pagan con tarjeta o billetera móvil. El cobro lo hace una empresa de pagos aparte, así que RutaColca no guarda datos de tarjetas. Antes de confirmar, el sistema revisa el boleto turístico del turista con AUTOCOLCA. Cuando la reserva queda lista, el guía recibe el itinerario y la lista de su grupo por WhatsApp, sin llamadas ni mensajes sueltos. Un administrador aprueba a las agencias nuevas y revisa cuánto vendió cada una por mes. Si hoy usted anota reservas en un cuaderno o las coordina por teléfono, todo eso quedaría en un solo lugar.

Parte B · Extensión

10. Paso 7: diagrama de despliegue en producción

El despliegue muestra dónde corre cada contenedor del Paso 3, en qué nodos y con qué infraestructura. Hicimos un diagrama para el entorno de producción; desarrollo y pruebas tendrían el suyo. Solo usamos instancias de los contenedores que ya existen en el modelo, sin crear contenedores nuevos.

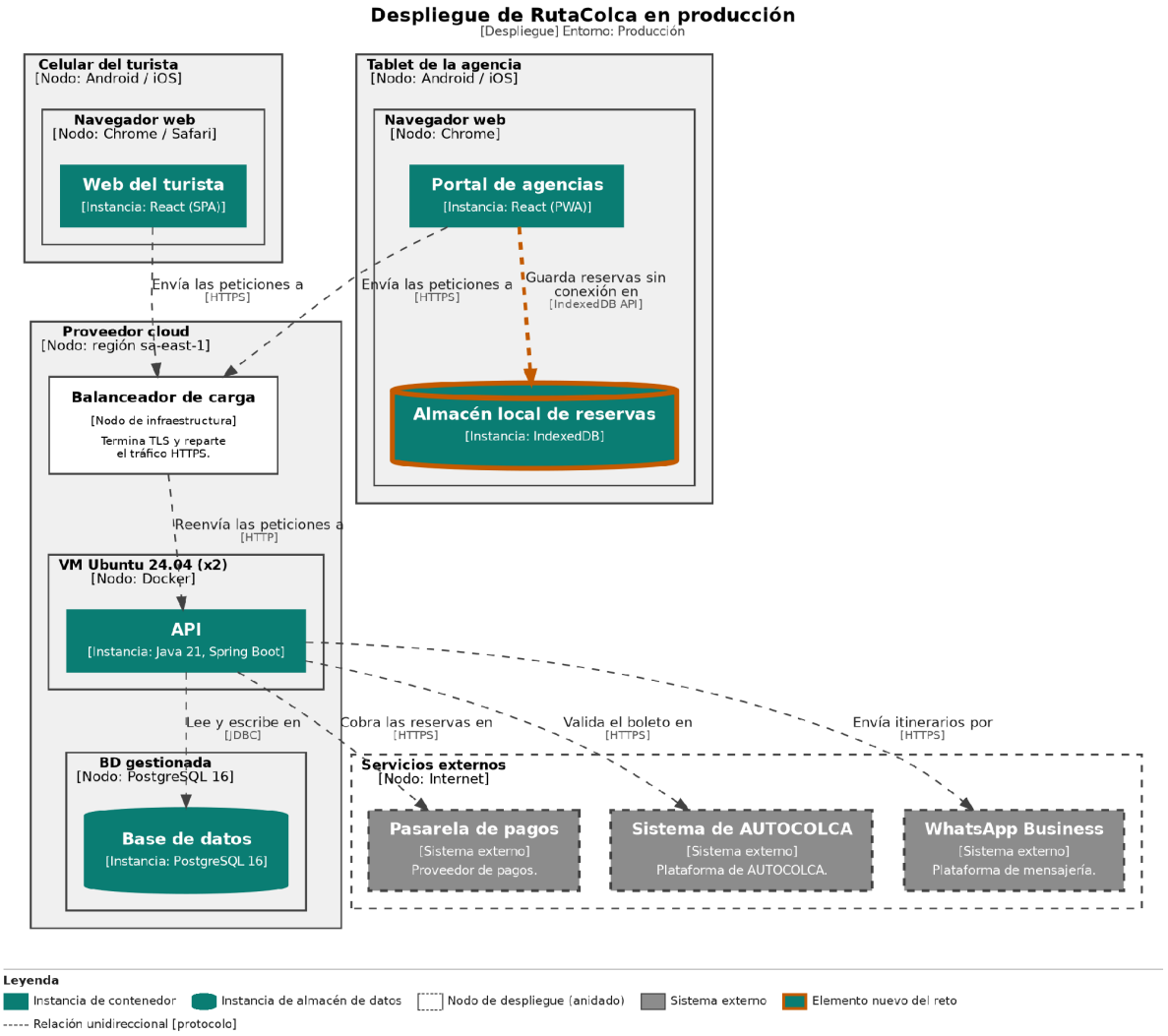


Figura 4. Despliegue de RutaColca en producción.

Tabla 12. Nodos del entorno de producción.

Nodo	Tecnología	Qué contiene
Celular del turista	Android / iOS, con navegador Chrome o Safari	Instancia de la web del turista (React SPA).
Tablet de la agencia	Android / iOS, con navegador Chrome	Instancia del portal de agencias (React PWA) y del almacén local de reservas (ver sección 11).
Proveedor cloud	Región sa-east-1	Balanceador de carga, las VM de la API y la base de datos gestionada.
Balanceador de carga	Nodo de infraestructura	Termina TLS y reparte el tráfico HTTPS hacia la API.
VM Ubuntu 24.04 (x2)	Ubuntu 24.04 con Docker	Instancia de la API (Java 21, Spring Boot) en dos máquinas.
BD gestionada	PostgreSQL 16 administrado	Instancia de la base de datos.
Servicios externos	Internet	Pasarela de pagos, AUTOCOLCA y WhatsApp Business, como sistemas de terceros.

La API corre en dos máquinas virtuales por la demanda: los picos de julio y agosto, y los feriados largos, pueden llenar una sola máquina. Con dos réplicas detrás del balanceador, si una falla la otra sigue atendiendo. En el modelo esto se indica con el número de instancias del nodo («VM Ubuntu 24.04 (x2)»).

En la figura dejamos solo el camino cliente → balanceador → API para que se lea mejor. Structurizr además dibuja la relación directa web → API que hereda del modelo de contenedores; no es un error, es la misma relación vista desde otro nivel.

11. Paso 8: reto de reservas sin conexión en Chivay (HU-05)

Las agencias de Chivay deben registrar reservas aunque no haya internet y sincronizarlas cuando vuelva la conexión. Respondimos las cuatro preguntas de la guía y reflejamos las respuestas en el modelo.

1. ¿Qué contenedor nuevo aparece?

Aparece el **Almacén local de reservas**. Es un almacén de datos, no una aplicación, y usa **IndexedDB**, la base de datos que trae el navegador. Vive dentro del propio portal de agencias, que ya es una PWA según el ADR-001, así que no se agrega ninguna tecnología nueva ni una app móvil aparte.

2. ¿Qué relación nueva tiene el portal y qué cambia en portal → API?

El portal tiene una relación nueva con el almacén: «Guarda reservas sin conexión en» [IndexedDB API]. La relación con la API cambia en su forma de uso. Antes el portal enviaba cada reserva en el momento; ahora, cuando no hay internet, las reservas esperan en el almacén y el portal las manda juntas al volver la conexión. Por eso agregamos una relación nueva, «Sincroniza reservas pendientes con» [JSON/HTTPS], además de la relación normal «Envía tours y reservas a».

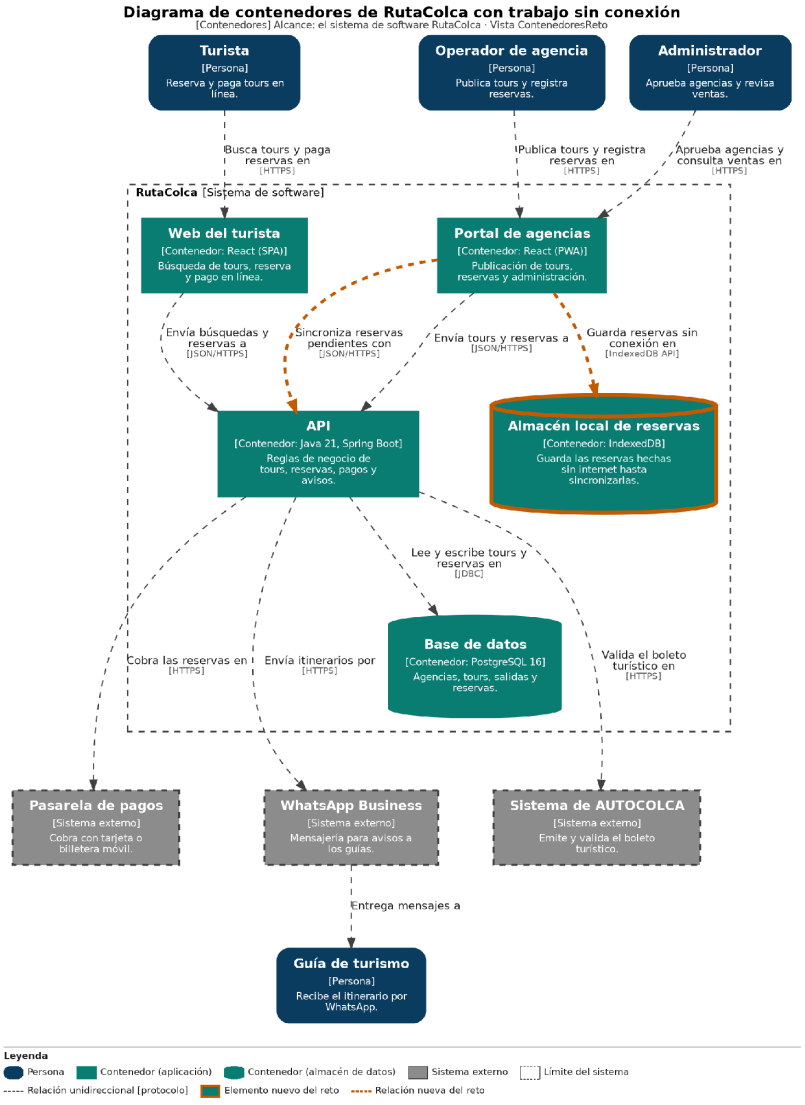


Figura 5. Vista ContenedoresReto: el reto sin conexión resuelto (en naranja lo nuevo).

3. ¿En qué nodo del despliegue se ejecuta el contenedor nuevo?

En la **tablet de la agencia**, dentro del nodo «Navegador web», junto a la instancia del portal de agencias. Se ve en la Figura 4. No necesita servidor propio porque los datos viven en el navegador del dispositivo.

4. ¿Qué riesgo de negocio introduce y qué regla proponemos?

El riesgo principal es la **sobreventa**: dos agencias venden el último cupo de una salida, una desde Chivay sin conexión y otra por la plataforma en línea. Como la agencia sin conexión no ve el cupo real, solo se enteraría del conflicto al sincronizar, cuando el turista ya está en el mostrador. Hay otros riesgos menores: reservas duplicadas si se reenvía el lote, y datos personales guardados en una tablet.

Proponemos tres reglas. Primera, cada agencia de Chivay tiene un **cupo reservado** por salida para ventas presenciales, y sin conexión solo puede vender de ese cupo. Segunda, el servidor tiene siempre la última palabra: al sincronizar, la API confirma cada reserva por orden de llegada y rechaza las que ya no tienen cupo, y el portal avisa al operador para que ubique al cliente en otra salida. Tercera, cada reserva lleva un identificador único creado en la tablet, de modo que reenviarla no la duplica.

Como supuesto, la validación del boleto turístico con AUTOCOLCA ocurre al sincronizar, porque sin internet no se puede consultar. Mientras tanto la reserva queda como PENDIENTE.

ADR-002: Trabajo sin conexión en las agencias de Chivay

Campo	Contenido
Estado	Aceptado por el grupo
Fecha	18 de octubre de 2026
Contexto	La historia HU-05 pide que las agencias de Chivay registren reservas presenciales aunque no haya internet. La conexión en Chivay es intermitente. El ADR-001 ya define el portal de agencias como una PWA en React, y el equipo es de 4 desarrolladores con un mes para el MVP, por lo que conviene no sumar tecnologías nuevas.
Decisión	El portal de agencias guarda las reservas presenciales en un almacén local (IndexedDB) de la tablet cuando no hay internet, con un Service Worker propio de la PWA. Al volver la conexión, las envía a la API en lote. Reglas: (1) cada agencia de Chivay tiene un cupo reservado por salida para ventas sin conexión; (2) la API decide el cupo por orden de llegada y rechaza las reservas sin cupo; (3) cada reserva lleva un identificador único generado en la tablet para evitar duplicados; (4) el boleto turístico se valida al sincronizar. No se usan apps nativas ni colas de mensajes, porque no forman parte del ADR-001.
Consecuencias	A favor: las agencias de Chivay pueden vender sin internet y no se agrega infraestructura nueva. En contra: el cupo reservado para ventas presenciales no se puede vender en línea; la sincronización añade lógica y pruebas; hay datos personales en la tablet, por lo que se guardan solo los mínimos (nombre, boleto y salida) y se borran al sincronizar. Cambios en el modelo: nuevo contenedor «Almacén local de reservas», dos relaciones nuevas del portal y una instancia nueva en el nodo «Tablet de la agencia».

12. Paso 9: exportar a C4-PlantUML y Mermaid

Una ventaja de escribir el modelo como código es que las otras notaciones salen del mismo archivo. Con la CLI de Structurizr validamos el modelo y generamos C4-PlantUML y Mermaid; después PlantUML genera los PNG. Estos son los comandos que usamos en el repositorio del grupo.

```
./structurizr.sh validate -w docs/architecture/c4/ruta-colca/workspace.dsl
./structurizr.sh export -w docs/architecture/c4/ruta-colca/workspace.dsl -f plantuml/c4plantuml -o
docs/architecture/c4/ruta-colca/export
./structurizr.sh export -w docs/architecture/c4/ruta-colca/workspace.dsl -f mermaid -o
docs/architecture/c4/ruta-colca/export
java -jar plantuml.jar -tpng -o ../img docs/architecture/c4/ruta-colca/export/*.puml
```

Para comparar, el diagrama de contexto de RutaColca queda así en las otras dos notaciones. Son las mismas personas, el mismo sistema y las mismas relaciones; cambia solo la sintaxis.

C4-PlantUML

```
@startuml
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Context.puml
title Diagrama de contexto de RutaColca
Person(turista, "Turista", "Busca, reserva y paga tours.")
Person(operador, "Operador de agencia", "Publica tours y registra reservas.")
Person(admin, "Administrador", "Aprueba agencias y revisa ventas.")
Person(guia, "Guía de turismo", "Recibe el itinerario por WhatsApp.")
System(ruta, "RutaColca", "Publicar, reservar y pagar tours del Colca.")
System_Ext(pasarela, "Pasarela de pagos", "Cobra con tarjeta o billetera móvil.")
System_Ext(autocolca, "Sistema de AUTOCOLCA", "Valida el boleto turístico.")
System_Ext(whatsapp, "WhatsApp Business", "Mensajería para avisos.")
Rel(turista, ruta, "Busca tours, reserva y paga en")
Rel(operador, ruta, "Publica tours y registra reservas en")
Rel(admin, ruta, "Aprueba agencias y revisa ventas en")
Rel(ruta, pasarela, "Cobra las reservas en")
Rel(ruta, autocolca, "Valida el boleto turístico en")
Rel(ruta, whatsapp, "Envía itinerarios por")
Rel(whatsapp, guia, "Entrega mensajes a")
SHOW_LEGEND()
@enduml
```

Mermaid

```
C4Context
  title Diagrama de contexto de RutaColca
```

```
Person(turista, "Turista", "Busca, reserva y paga tours.")
Person(operador, "Operador de agencia", "Publica tours y registra reservas.")
Person(admin, "Administrador", "Aprueba agencias y revisa ventas.")
Person(guia, "Guía de turismo", "Recibe el itinerario por WhatsApp.")
System(ruta, "RutaColca", "Publicar, reservar y pagar tours del Colca.")
System_Ext(pasarela, "Pasarela de pagos", "Cobra con tarjeta o billetera móvil.")
System_Ext(autocolca, "Sistema de AUTOCOLCA", "Valida el boleto turístico.")
System_Ext(whatsapp, "WhatsApp Business", "Mensajería para avisos.")
Rel(turista, ruta, "Busca tours, reserva y paga en")
Rel(operador, ruta, "Publica tours y registra reservas en")
Rel(admin, ruta, "Aprueba agencias y revisa ventas en")
Rel(ruta, pasarela, "Cobra las reservas en")
Rel(ruta, autocolca, "Valida el boleto turístico en")
Rel(ruta, whatsapp, "Envía itinerarios por")
Rel(whatsapp, guia, "Entrega mensajes a")
```

Tabla 13. Comparación de las tres notaciones.

Aspecto	Structurizr DSL	C4-PlantUML	Mermaid
Idea central	Un solo modelo; las vistas se generan de él.	Un archivo por diagrama.	Un bloque de texto por diagrama.
Si renombramos un elemento	Se cambia una vez y se actualizan todas las vistas.	Hay que editar cada diagrama.	Hay que editar cada diagrama.
Integración	Exporta a PlantUML y Mermaid.	Rápido y conocido.	Se muestra directo en Markdown y GitHub.
Lo usaríamos para	Documentación viva del sistema.	Diagramas sueltos.	Documentación ligera en el repositorio.

13. Paso 10: bitácora de uso de IA

Registramos cada prompt usado (P1 a P8), lo que aceptamos, lo que rechazamos o corregimos y quién lo verificó. La regla fue la misma en todos los casos: ninguna respuesta de la IA entró al modelo sin pasar por la lista de revisión.

Tabla 14. Bitácora de uso de IA (bitacora-ia.md).

N.º	Fecha	Prompt	Qué se aceptó	Qué se rechazó o corrigió	Responsable
1	15/10/2026	P1 · Revisar el inventario	Confirmó que 11 de 12 elementos estaban bien clasificados.	Dijo que el guía no debía ser persona porque no usa la plataforma. Lo rechazamos: recibe el itinerario y es parte del flujo.	Rodrigo Estefanero
2	15/10/2026	P2 · Contexto en Structurizr DSL	Bloque model con personas, sistemas externos con etiqueta «Externo» y relaciones.	Una relación decía «Usa AUTOCOLCA»; la cambiamos por «Valida el boleto turístico en». Faltaba la relación WhatsApp → guía.	Renzo Mamani
3	15/10/2026	P3 · Contenedores a partir del ADR	Los cuatro contenedores con su tecnología y las relaciones con protocolo.	Agregó un contenedor «Docker» y otro de caché. Los quitamos: Docker no es contenedor C4 y la caché no está en el ADR-001. También movió una relación portal → base de datos; la pasamos a portal → API.	Renzo Mamani
4	15/10/2026	P4 · Vista dinámica desde una historia	Numeración y etiquetas con la acción concreta.	Cobraba antes de validar el boleto; invertimos el orden. Indicó que faltaba API → WhatsApp en contenedores y la agregamos.	Rodrigo Estefanero
5	15/10/2026	P5 · Auditor C4	Hallazgos sobre siglas sin explicar y dirección de la relación con el guía.	Propuso un contenedor «Docker»; lo rechazamos con la regla de abstracciones de C4.	Christian Yana

N.º	Fecha	Prompt	Qué se aceptó	Qué se rechazó o corrigió	Responsable
6	15/10/2026	P6 · Borrador de explicación	La estructura (qué hace, quién lo usa, con qué se conecta) y el ejemplo del cuaderno de reservas.	Usaba las palabras «plataforma en la nube» y «API». El vocero reescribió el texto con sus palabras y lo ajustó a unas 120 palabras.	Sergio Estrada
7	18/10/2026	P7 · Despliegue por entorno	Nodos del celular, la tablet y el proveedor cloud; API en 2 VM; balanceador como nodo de infraestructura.	Creó un contenedor nuevo para Docker dentro de la vista; lo eliminamos. Las relaciones entre nodos estaban dentro del bloque del entorno y las movimos afuera.	Rodrigo Estefanero
8	18/10/2026	P8 · Analizar el impacto del reto	Cinco preguntas útiles: cupo, duplicados, datos guardados en la tablet, cuándo se valida el boleto y cómo se sabe que volvió la conexión.	Sugirió una cola de mensajes y una app nativa. No están en el ADR-001 y quedaron fuera.	Christian Yana

14. Entregables

Tabla 15. Entregables y dónde está cada uno.

Código	Archivo en el repositorio	Parte	En este documento
E1	docs/architecture/c4/ruta-colca/inventario.md	A	Sección 4
E2	docs/architecture/c4/ruta-colca/workspace.dsl (contexto, contenedores, dinámico)	A	Secciones 5 a 7 y Anexo B
E3	revision-cruzada.md y explicacion.md	A	Secciones 8 y 9
E4	workspace.dsl con despliegue y vista ContenedoresReto	B	Secciones 10 y 11 y Anexo B
E5	docs/architecture/adr/ADR-002-trabajo-sin-conexion.md	B	Sección 11
E6	docs/architecture/c4/ruta-colca/export/ (C4-PlantUML, Mermaid y PNG)	B	Sección 12
E7	docs/architecture/c4/ruta-colca/bitacora-ia.md	A y B	Sección 13

14.1 Autoevaluación con la rúbrica

Tabla 16. Dónde cumplimos cada criterio de la rúbrica de 20 puntos.

Criterio	Cómo lo cumplimos
Contexto	Cuatro personas y tres sistemas externos con descripción; RutaColca es una sola caja (Figura 1).
Contenedores y despliegue	Tecnología en cada contenedor, ninguno es Docker; despliegue con nodos, instancias e infraestructura; reto resuelto (Figuras 2, 4 y 5).
Relaciones	Unidireccionales, con verbo específico y protocolo entre contenedores (Tabla 9).
Dinámico y notación	Pasos numerados y válidos, título, tipos y leyenda en cada vista; exportación a otras notaciones (Figura 3 y sección 12).

Criterio	Cómo lo cumplimos
Comunicación y trazabilidad	Explicación sin jerga (sección 9), bitácora de IA completa (sección 13) y revisión cruzada documentada (sección 8).

15. Preguntas para la socialización

1. **¿Por qué RutaColca aparece como una sola caja en el contexto y como un límite con varios contenedores en el siguiente nivel?** Porque cada nivel responde una pregunta distinta para una audiencia distinta. El contexto dice quién usa el sistema y con qué se conecta, y lo puede entender cualquiera, así que el sistema se muestra sin detalles. Al hacer zoom, el contenedor-límite deja ver las aplicaciones y los datos que lo forman, que interesan al equipo técnico.
2. **¿Qué diagrama le mostrarían a la gerente de una agencia y cuál al equipo de operaciones?** A la gerente, el diagrama de contexto: no necesita tecnologías, solo saber qué hace el sistema, quién lo usa y con qué servicios se conecta. Al equipo de operaciones, el de despliegue, porque muestra en qué nodos corre cada parte, las réplicas y el balanceador; con el de contenedores completa el panorama.
3. **¿Qué propuesta de la IA rechazaron y qué regla de C4 usaron?** Rechazamos el contenedor «Docker», con la regla de que un contenedor C4 es una aplicación o un almacén de datos. También quitamos Redis, la cola de mensajes y la app nativa porque no están en el ADR-001. Y cambiamos el orden del cobro y la validación del boleto, que fue una decisión de negocio del grupo.
4. **¿Qué cambió en su modelo al resolver el reto sin conexión y en qué diagramas se nota?** Se nota en dos diagramas: en el de contenedores aparece el almacén local y dos relaciones nuevas del portal (Figura 5), y en el de despliegue el almacén aparece en la tablet de la agencia (Figura 4). El contexto y el dinámico no cambian, porque el reto no agrega personas ni sistemas externos ni altera el escenario de reservar y pagar en línea.

16. Conclusiones

Modelar RutaColca nos mostró que cada nivel de C4 sirve para una conversación distinta, y que mezclar niveles (por ejemplo, poner la base de datos dentro del contexto) confunde más de lo que ayuda. Escribir el modelo una sola vez y generar las vistas desde ahí nos ahorró inconsistencias: al agregar el almacén local para el reto no tuvimos que redibujar nada, solo actualizar el modelo.

Sobre la IA, nos fue útil para sacar borradores rápidos, pero se equivocó en cosas que la lista de revisión detectó enseguida, como el contenedor Docker, el orden del cobro y relaciones con «usa». La IA propone y el equipo verifica.

El reto sin conexión dejó una lección de negocio: la parte técnica (guardar en IndexedDB y sincronizar) es la más sencilla; lo difícil es decidir qué pasa con el último cupo. Esa decisión quedó escrita en el ADR-002.

Anexo A. Lista de revisión aplicada a nuestros diagramas

Resumen de la lista oficial de c4model.com aplicada a los cuatro diagramas del modelo.

Tabla 17. Lista de revisión de un diagrama C4 aplicada a RutaColca.

Aspecto	Pregunta de verificación	Resultado en RutaColca
General	¿Tiene un título que indique tipo y alcance? ¿Se entiende el alcance? ¿Tiene leyenda?	Cumple. Cada figura tiene título, subtítulo con el tipo y el alcance, y leyenda al pie.
Elementos	¿Cada elemento tiene nombre, tipo explícito y una descripción breve? ¿Contenedores y componentes indican tecnología?	Cumple. Todos muestran su tipo entre corchetes y una descripción; los contenedores indican su tecnología.
Siglas	¿Las siglas (SEE, PWA) se entienden o están en la leyenda?	Cumple. SPA, PWA y AUTOCOLCA se explican en el texto del documento (sección 4).
Relaciones	¿Cada línea tiene etiqueta que describe la intención y coincide con la dirección? ¿Entre contenedores indica protocolo?	Cumple. Todas las flechas tienen verbo y una sola dirección; las de contenedores llevan protocolo.
Notación	¿Se explican formas, colores, bordes, líneas y puntas de flecha? ¿Funciona en blanco y negro?	Cumple. La leyenda explica formas y bordes; las personas son rectángulos redondeados, la base de datos un cilindro y los externos tienen borde discontinuo.
Audiencia	¿Lo entiende alguien que no estuvo en la reunión, sin que se lo expliquen?	Cumple en el contexto, comprobado con la explicación para la gerente (sección 9).

Anexo B. workspace.dsl completo

Archivo docs/architecture/c4/ruta-colca/workspace.dsl, con el modelo, las cinco vistas (Contexto, Contenedores, Dinámico, Despliegue y ContenedoresReto) y los estilos. La vista Contenedores excluye el almacén local y la relación de sincronización, para que conserve el estado de la Parte A.

```
workspace "RutaColca" "Modelo C4 de RutaColca - Grupo N.° 3" {

    !identifiers hierarchical

    model {
        // PASO 1 y 2 · Personas
        turista = person "Turista" "Visitante que busca tours al valle del Colca, reserva una salida
y paga en línea."
        operador = person "Operador de agencia" "Persona de una agencia de Arequipa o Chivay que
publica tours y registra reservas."
        guia = person "Guía de turismo" "Acompaña a los grupos. Recibe el itinerario y la lista del
grupo por WhatsApp."
        admin = person "Administrador" "Aprueba a las agencias que se registran y revisa las
ventas."

        // PASO 2 · Sistemas externos (etiqueta "Externo")
        pasarela = softwareSystem "Pasarela de pagos" "Cobra con tarjeta o billetera móvil.
RutaColca no guarda datos de tarjetas." "Externo"
        autocolca = softwareSystem "Sistema de AUTOCOLCA" "Emite y valida el boleto turístico del
Colca (supuesto didáctico, API HTTPS)." "Externo"
        whatsapp = softwareSystem "WhatsApp Business" "Mensajería para enviar avisos a los guías."
"Externo"

        // PASO 2 y 3 · El sistema en estudio y sus contenedores (ADR-001)
        ruta = softwareSystem "RutaColca" "Permite publicar tours del valle del Colca, reservarlos y
pagarlos en línea." {
            web = container "Web del turista" "Búsqueda de tours, reserva y pago en línea." "React
(SPA)"
            portal = container "Portal de agencias" "Publicación de tours, registro de reservas y
administración." "React (PWA)"
            api = container "API" "Reglas de negocio de tours, reservas, pagos y avisos." "Java 21,
Spring Boot"
            bd = container "Base de datos" "Agencias, tours, salidas y reservas." "PostgreSQL 16"
"Database"
            // PASO 8 · Reto sin conexión (ADR-002)
            almacen = container "Almacén local de reservas" "Guarda las reservas hechas sin internet
hasta sincronizarlas." "IndexedDB" "Database,Nuevo"
        }
```

```
        // PASO 2 · Relaciones del contexto (sin protocolo)
        turista -> ruta "Busca tours, reserva y paga en"
        operador -> ruta "Publica tours y registra reservas en"
        admin -> ruta "Aprueba agencias y revisa ventas en"
        ruta -> pasarela "Cobra las reservas en"
        ruta -> autocolca "Valida el boleto turístico en"
        ruta -> whatsapp "Envía itinerarios por"
        whatsapp -> guia "Entrega mensajes a"

        // PASO 3 · Relaciones entre contenedores (con protocolo)
        turista -> ruta.web "Busca tours y paga reservas en" "HTTPS"
        operador -> ruta.portal "Publica tours y registra reservas en" "HTTPS"
        admin -> ruta.portal "Aprueba agencias y consulta ventas en" "HTTPS"
        ruta.web -> ruta.api "Envía búsquedas y reservas a" "JSON/HTTPS"
        ruta.portal -> ruta.api "Envía tours y reservas a" "JSON/HTTPS"
        ruta.api -> ruta.bd "Lee y escribe tours y reservas en" "JDBC"
        ruta.api -> pasarela "Cobra las reservas en" "HTTPS"
        ruta.api -> autocolca "Valida el boleto turístico en" "HTTPS"
        ruta.api -> whatsapp "Envía itinerarios por" "HTTPS"

        // PASO 8 · Relaciones nuevas del reto
        ruta.portal -> ruta.almacen "Guarda reservas sin conexión en" "IndexedDB API"
        ruta.portal -> ruta.api "Sincroniza reservas pendientes con" "JSON/HTTPS" "Sincronizacion"

        // PASO 7 · Despliegue en producción
        produccion = deploymentEnvironment "Produccion" {
            celular = deploymentNode "Celular del turista" "Dispositivo del turista" "Android / iOS"
{
                navegador = deploymentNode "Navegador web" "Navegador del celular" "Chrome / Safari"
{
                    web = containerInstance ruta.web
                }
            }
            tablet = deploymentNode "Tablet de la agencia" "Dispositivo de la agencia" "Android /
iOS" {
                navegador = deploymentNode "Navegador web" "Navegador de la tablet" "Chrome" {
                    portal = containerInstance ruta.portal
                    almacen = containerInstance ruta.almacen
                }
            }
            nube = deploymentNode "Proveedor cloud" "Infraestructura en la nube" "Región sa-east-1"
{
                balanceador = infrastructureNode "Balanceador de carga" "Termina TLS y reparte el
tráfico HTTPS." "Balanceador gestionado"
                vm = deploymentNode "VM Ubuntu 24.04" "Máquina virtual que ejecuta la API con
Docker" "Ubuntu 24.04, Docker" "" 2 {
                    api = containerInstance ruta.api
                }
                bdgestionada = deploymentNode "BD gestionada" "Servicio de base de datos
administrado" "PostgreSQL 16" {
                    bd = containerInstance ruta.bd
                }
            }
            externos = deploymentNode "Servicios externos" "Proveedores de terceros" "Internet" {
                pasarelaInst = softwareSystemInstance pasarela
                autocolcaInst = softwareSystemInstance autocolca
                whatsappInst = softwareSystemInstance whatsapp
            }
        }

        // Relaciones entre nodos: van fuera del bloque del entorno
        produccion.celular.navegador.web -> produccion.nube.balanceador "Envía las peticiones a"
"HTTPS"
        produccion.tablet.navegador.portal -> produccion.nube.balanceador "Envía las peticiones a"
"HTTPS"
        produccion.nube.balanceador -> produccion.nube.vm.api "Reenvía las peticiones a" "HTTP"
    }

    views {
        systemContext ruta "Contexto" "Diagrama de contexto de RutaColca" {
            title "Diagrama de contexto de RutaColca"
            include *
            include guia
            autolayout lr
        }

        container ruta "Contenedores" "Diagrama de contenedores de RutaColca" {
            title "Diagrama de contenedores de RutaColca"
            include *
            include guia
            exclude ruta.almacen
            exclude "relationship.tag==Sincronizacion"
            autolayout tb
        }

        // PASO 4 · Dinámico. Se valida el boleto ANTES de cobrar.
        dynamic ruta "Dinamico" "Reservar y pagar un tour" {
            title "Diagrama dinámico: Reservar y pagar un tour (HU-02 y HU-03)"
            turista -> ruta.web "Elige la salida, ingresa su boleto y confirma el pago"
            ruta.web -> ruta.api "Envía la reserva y el número de boleto"
```

```
            ruta.api -> autocolca "Valida el boleto turístico"
            ruta.api -> pasarela "Cobra el monto de la reserva"
            ruta.api -> ruta.bd "Guarda la reserva como CONFIRMADA"
            ruta.api -> whatsapp "Envía el itinerario y la lista del grupo"
            autolayout tb
        }

        // PASO 7 · Despliegue
        deployment ruta "Produccion" "DespliegueProduccion" "Despliegue de RutaColca en producción"
{
            title "Despliegue de RutaColca en producción"
            include *
            autolayout lr
        }

        // PASO 8 · Segunda vista de contenedores (reto sin conexión)
        container ruta "ContenedoresReto" "Contenedores de RutaColca con trabajo sin conexión" {
            title "Diagrama de contenedores de RutaColca con trabajo sin conexión"
            include *
            include guia
            autolayout tb
        }

        styles {
            element "Person" {
                shape Person
                background #0B3D62
                color #ffffff
            }
            element "Software System" {
                background #1B5E8B
                color #ffffff
            }
            element "Container" {
                background #0E7D73
                color #ffffff
            }
            element "Externo" {
                background #8C8C8C
                color #ffffff
            }
            element "Database" {
                shape Cylinder
            }
            element "Nuevo" {
                stroke #C65D00
                strokeWidth 6
            }
            relationship "Sincronizacion" {
                color #C65D00
                thickness 4
            }
        }
    }
}
```

Referencias

[1] S. Brown, “The C4 model for visualising software architecture,” c4model.com. [En línea]. Disponible en: https://c4model.com/ (consultado el 11 de oct. de 2026).

[2] S. Brown, The C4 Model: Visualizing Software Architecture. Santa Rosa, CA, EE. UU.: O’Reilly Media, 2026, ISBN 979-8-341-66012-0.

[3] Structurizr, “Structurizr CLI,” repositorio de GitHub. [En línea]. Disponible en: https://github.com/structurizr/cli (consultado el 11 de oct. de 2026).

[4] Structurizr, “Structurizr DSL: Language reference.” [En línea]. Disponible en: https://docs.structurizr.com/dsl/language